

Parameters, stack and scope

CSC103 Programming Fundamentals / Lecture 16

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Trace main with $x=4$ calling `addOne(x)`, storing its returned result in `y`. Show the caller and callee values before and after return.

Keep `x` in the caller and pass its value to `addOne`.

Learning objectives

- Explain pass-by-value and stack frames
- Resolve overloaded calls
- Identify local and block scope

CLO-2

Pass-by-value

Call stack and activation records

Method overloading

Variable scope

Pass-by-value

- Java copies each argument value into a parameter.
- Reassigning a primitive parameter does not reassign the caller variable.
- Reference values also pass by value.

Example

```
static void change(int n) {  
    n = 99;  
}  
// change(x) leaves x unchanged
```

Call stack and activation records

- Each active invocation has a stack frame with its own local state.
- A call suspends the caller until the called method returns normally or throws.
- A return resumes the caller at the call site.

Example

```
main calls doubleIt(4)
doubleIt has its own parameter n=4
doubleIt returns 8
main stores 8
```

Method overloading

- Overloads share a name but use different parameter lists.
- The compiler selects an applicable overload from argument types.
- Return type alone cannot distinguish overloads.

Example

```
static int twice(int n) { return n * 2; }  
static double twice(double n) { return n * 2; }
```

Variable scope

- A local name is available only within its scope.
- A block variable cannot be used outside its block.
- A local variable can hide a field with the same name.

Example

```
if (ready) {  
    int result = 10;  
}  
// result is unavailable here
```

Each invocation owns its local variables

- A caller's `x` and a callee's `n` are distinct variables.
- Parameter binding copies the argument value into the callee frame.
- The returned value travels back separately and can be stored in another caller variable.

Example

Caller: `x=4`

Callee: `n` receives 4, returns 5

Caller after `return`: `x=4`, `y=5`

Overloading uses the declared parameter list

- Methods with the same name can accept different argument counts or types.
- The compiler resolves the call using the argument types available at compilation.
- A different return type alone does not create a new overload.

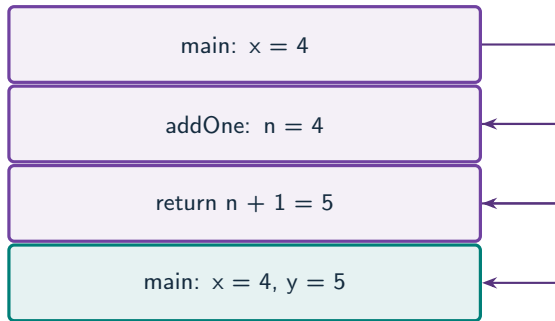
Example

```
max(int a, int b)
```

```
max(int a, int b, int c)
```

These are distinct signatures.

A call has its own local state



What to notice

The parameter is a separate variable. Returning a value does not overwrite the argument.

Key distinctions

Property	Scope	Lifetime
Question	Where can a name be used?	How long does state exist?
Local variable	Its enclosing declaration scope	During the invocation
Block variable	Inside the relevant block	During block execution

These distinctions determine which operation or control structure fits the problem.

First example: methods

```
static void change(int n) { n = 99; }  
static int twice(int n) { return n * 2; }  
static double twice(double n) { return n * 2; }
```

First example: execution

```
int x = 5;  
change(x);  
System.out.println(x);  
System.out.println(twice(3));  
System.out.println(twice(3.0));
```

First example: result

5

6

6.0

change modifies only its parameter. The argument type selects each twice overload.

Method overloading
Variable scope

Guided practice

Trace main with $x=4$ calling `addOne(x)`, storing its returned result in `y`. Show the caller and callee values before and after return.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Keep x in the caller and pass its value to `addOne`.
- Return $n + 1$ from the callee.
- Store the result in y and print both caller variables.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution16 {  
    public static void main(String[] args) throws Exception {  
        int x = 4;  
        int y = addOne(x);  
        System.out.println(x);  
        System.out.println(y);  
    }  
    static int addOne(int n) {  
        return n + 1;  
    }  
}
```

Execution trace

Moment	main frame	addOne frame
Before call	x=4, y not assigned	No frame yet
Inside call	x=4, waiting	n=4
At return	Waiting for result	Returns 5
After return	x=4, y=5	Invocation completed

Follow the changing state in execution order.

Solution result

4

5

Why this result follows
Store the result in y and print both
caller variables.

A common incorrect approach

```
static int f(int x) { return x; }  
static double f(int x) { return x; }
```

Example

Both declarations have the same name and parameter types. Change the parameter list or use a different method name.

Boundary and failure checks

Check the stated input assumptions.

Write overloaded max methods for two ints and three ints.

Reuse the two-argument version inside the three-argument version.

Caller $x=4$. Callee parameter $n=4$.

addOne returns $n+1$, which is 5.

Caller $y=5$ and x remains 4.

Check your understanding

Does changing a parameter change its primitive argument?
Is a for-header variable visible after the loop?

Write a prediction and explain the rule that supports it.

Answer and explanation

No. The parameter holds a copy. A variable declared in the for header is outside scope after that loop.

Both declarations have the same name and parameter types. Change the parameter list or use a different method name.

Compare cases and predict outcomes

Moment	Caller x	Callee n
Before call	4	Not active
After $n = n + 1$	4	5
After return	4	Invocation ended

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Trace `bump(x)` when `bump` increments its parameter and returns it, but `main` ignores the return value. How must `main` change to update `x`?

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
static int bump(int n) {  
    return n + 1;  
}  
// In main:  
int x = 4;  
bump(x);          // returned 5 is ignored  
x = bump(x);      // now x is 5
```

- The first call leaves x equal to 4.
- The second call computes from that same value and returns 5.
- Assignment in the caller is what stores the new value.

Lecture summary

- pass-by-value and stack frames
- overloaded calls
- local and block scope
- Keep x in the caller and pass its value to `addOne`.
- Return $n + 1$ from the callee.
- Store the result in y and print both caller variables.

After-class practice

Write overloaded max methods for two ints and three ints. Reuse the two-argument version inside the three-argument version.

Syllabus reading

Deitel Ch 6

Next lecture: Midterm concept review